

FIN-220 Hands-On Machine Learning

HW 6

Due on December 01, 2025

1. Let $K : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$, be defined by $K(a,b) = (a^T b)^3$. Find $\varphi : \mathbb{R}^2 \rightarrow \mathbb{R}^4$ such that $\langle \varphi(a), \varphi(b) \rangle = K(a,b)$

$$(a^T b)^3$$

$$(a_1 b_1)^3 + 3(a_1 b_1)^2 a_2 b_2 + 3(a_1 b_1)(a_2 b_2)^2 + (a_2 b_2)^3$$

$$a_1^3 b_1^3 + 3a_1^2 a_2 b_1^2 b_2 + 3a_1 a_2^2 b_1 b_2^2 + a_2^3 b_2^3$$

$$(a_1^3)(b_1^3) + (\sqrt{3}a_1^2 a_2)(\sqrt{3}b_1^2 b_2) + (\sqrt{3}a_1 a_2^2)(\sqrt{3}b_1 b_2^2) + (a_2^3)(b_2^3)$$

So:

$$\varphi(a) = [a_1^3, \sqrt{3}a_1^2 a_2, \sqrt{3}a_1 a_2^2, a_2^3]$$

$$\varphi(b) = [b_1^3, \sqrt{3}b_1^2 b_2, \sqrt{3}b_1 b_2^2, b_2^3]$$

2. Consider the following dataset with one feature x and binary label $y \in \{0,1\}$:

Sample	x	y
1	2	0
2	3	0
3	10	1
4	12	1

The possible split thresholds (midpoints between sorted feature values) are:

$$t \in \{2.5, 6.5, 11\}.$$

Compute the Gini impurity for each split! Which one should you choose to split the root node?

Split 2.5

Left $x \leq 2.5$, $n = 1$, Sample 1 $\rightarrow y = [0]$

$$1 - (1/1)^2 - (0/1)^2$$

Right $x > 2.5$: $n = 3$, Samples 2,3,4 $\rightarrow y = [0,1,1]$

$$1 - (1/3)^2 - (2/3)^2 = 4/9 = 0.444$$

$$\text{Split } \frac{1}{4} * 0 + \frac{3}{4} * 4/9 = 3/9 = 0.333$$

Split 6.5

Left $x \leq 6.5$, $n = 2$, Samples 1,2 $\rightarrow y = [0,0]$

$$1 - (2/2)^2 - (0/2)^2 = 1 - 1 = 0$$

Right $x > 6.5$, $n = 2$, Samples 3, 5 $\rightarrow y = [1,1]$

$$1 - (0/2)^2 - (2/2)^2 = 1 - 1 = 0$$

$$\text{Split } 2/4 * 0 + 2/4 * 0 = 0$$

Split 11

Left $x \leq 11$, $n = 3$, Samples 1,2,3 $\rightarrow y = [0,0,1]$

$$1 - (2/3)^2 - (1/3)^2 = 4/9 = 0.444$$

Right $x > 11$, $n = 1$, Sample 4 $\rightarrow y = [1]$

$$1 - (0/1)^2 - (1/1)^2 = 0$$

$$\text{Split } \frac{3}{4} * 4/9 + \frac{1}{4} * 0 = 0.333$$

Minimum Gini for 6.5

3. Train and fine-tune a decision tree for the moons dataset by following these steps:

- Use `make_moons(n samples=10000, noise=0.4)` to generate a moons dataset.
- Use `train_test_split()` to split the dataset into a training set and a test set.

Code is in:

https://github.com/McSavage/FIN220/blob/main/lectures/HandsOn_Chapter6/06_scripts.ipynb

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Generate moons dataset
X, y = make_moons(n_samples=10000, noise=0.4, random_state=42)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

print(f"Training set size: {len(X_train)}")
print(f"Test set size: {len(X_test)}")
```

Training set size: 8000

Test set size: 2000

- Use grid search with cross-validation (with the help of the GridSearchCV class) to find good hyperparameter values for a DecisionTreeClassifier. Hint: try various values for max leaf nodes.
- Train it on the full training set using these hyperparameters, and measure your model's performance on the test set. You should get roughly 85% to 87% accuracy.

```
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'max_leaf_nodes': [2, 5, 10, 20, 50, 100, 200, 500, 1000],
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 5, 10, 20]
}

# Create a DecisionTreeClassifier
tree_clf = DecisionTreeClassifier(random_state=42)

# Perform grid search with cross-validation
# Note: n_jobs=1 to avoid output duplication in Jupyter
grid_search = GridSearchCV(tree_clf, param_grid, cv=5,
                           scoring='accuracy', verbose=1, n_jobs=1)

# Fit the grid search to the training data
grid_search.fit(X_train, y_train)

# Display the best parameters and best score in a single print
results = f"""
Best parameters: {grid_search.best_params_}
Best cross-validation score: {grid_search.best_score_:.4f}
Test set score: {grid_search.best_estimator_.score(X_test, y_test):.4f}
"""
print(results)
```

Fitting 5 folds for each of 180 candidates, totalling 900 fits

Best parameters: {'max_depth': None, 'max_leaf_nodes': 20, 'min_samples_split': 2}

Best cross-validation score: 0.8586

Test set score: 0.8700

```

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

# Get the best model from grid search
best_tree = grid_search.best_estimator_

# Create a mesh to plot decision boundaries
x_min, x_max = X_train[:, 0].min() - 0.5, X_train[:, 0].max() + 0.5
y_min, y_max = X_train[:, 1].min() - 0.5, X_train[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

# Predict on the mesh
Z = best_tree.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

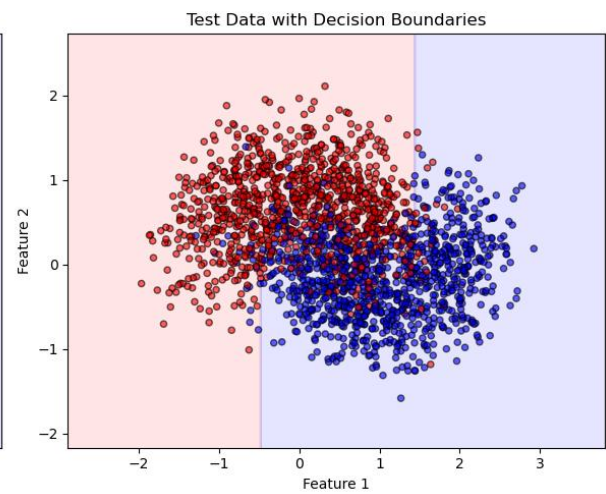
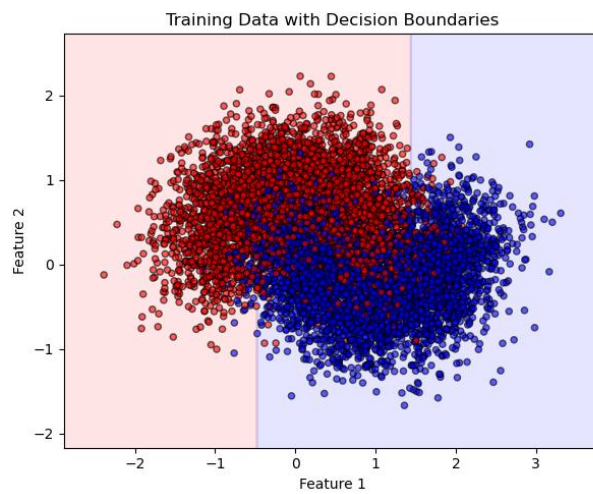
# Create the plot
plt.figure(figsize=(12, 5))

# Plot 1: Training data with decision boundaries
plt.subplot(1, 2, 1)
plt.contourf(xx, yy, Z, alpha=0.3, cmap=ListedColormap(['#FFAAAA',
                                                         '#AAAAFF']))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
            cmap=ListedColormap(['#FF0000', '#0000FF']),
            edgecolors='k', s=20, alpha=0.6)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Training Data with Decision Boundaries')

# Plot 2: Test data with decision boundaries
plt.subplot(1, 2, 2)
plt.contourf(xx, yy, Z, alpha=0.3, cmap=ListedColormap(['#FFAAAA',
                                                         '#AAAAFF']))
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test,
            cmap=ListedColormap(['#FF0000', '#0000FF']),
            edgecolors='k', s=20, alpha=0.6)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Test Data with Decision Boundaries')

plt.tight_layout()
plt.show()

```



```
# Check the actual properties of the best tree
print("Actual tree properties:")
print(f"Max depth reached: {best_tree.get_depth()}")
print(f"Number of leaves: {best_tree.get_n_leaves()}")
print(f"Total number of nodes: {best_tree.tree_.node_count}")
```

Actual tree properties:
Max depth reached: 7
Number of leaves: 20
Total number of nodes: 39